

Notes on Curves and Jacobians

Blair Azzopardi

October 6, 2026

Contents

1	Curves & LA	1
1.1	Risk	1
1.1.1	DV01	2
1.1.2	Forward Risk	3
1.1.3	Representation Flows $\frac{\partial \Pi}{\partial v_j}$	4
1.1.4	Auto differentiation	6
1.1.5	Instrument Sensitivities $\frac{\partial r_i}{\partial v_j}$	6
1.1.6	Curve solver	6
2	References	17

1 Curves & LA

1.1 Risk

Effective trading requires understanding how portfolio valuations react to market variables. Because we utilize tradable instruments to hedge against downside exposure, we also need to measure our portfolio's sensitivity to those specific trades. We refer to these sensitivities as risk.

A portfolio's value Π can be expressed as a weighted sum of its underlying instruments π_i .

$$\begin{aligned}\Pi &= \sum_i w_i \pi_i \\ &= \vec{w} \cdot \vec{\pi} \\ &= w_i \pi_i\end{aligned}$$

where the last equality uses Einstein summation notation where repeated indices (in this case i) are summed over all possible values. We will use this notation throughout this document (unless otherwise specified).

Each instrument's value π_i can be expressed as a sum of its discounted cashflows c_{ij} . We use v_{ij} for the discount factor.

So we can express the instrument value as:

$$\pi_i = c_{ij} v_{ij}$$

To value a portfolio we can simply sum up all of the cashflows over all the instruments.

$$\begin{aligned}\Pi &= w_i \pi_i \\ &= w_i c_{ij} v_{ij}\end{aligned}$$

We can aggregate all cashflows and discount factors by date, scale each cashflow by its corresponding weight and express the portfolio value as a sum of discounted cashflows:

$$\Pi = c_j v_j$$

1.1.1 DV01

When we look at risk with respect to some vector of tradeable instruments s_i , we scale by $\frac{1}{100}$ to consider the basis point risk, since the instruments are quoted in percent.

$$R_i^{\text{inst}} = \frac{1}{100} \frac{\partial \Pi}{\partial s_i}$$

We can use the chain rule to express this in terms of sensitivities to the discount factors v_i :

$$\begin{aligned}R_i^{\text{inst}} &= \frac{1}{100} \frac{\partial v_j}{\partial s_i} \frac{\partial \Pi}{\partial v_j} \\ &= \frac{1}{100} \left(\frac{\partial s_i}{\partial v_j} \right)^{-1} \frac{\partial \Pi}{\partial v_j} \\ &= \frac{1}{100} \left(\frac{\partial r_i}{\partial v_j} \right)^{-1} \frac{\partial \Pi}{\partial v_j}\end{aligned}$$

where j (nodes) is summed over since it's a repeated index and we're using Einstein summation notation.

The third line applies the inverse function theorem to the calibration map $\vec{v} \leftrightarrow \vec{s}$, and the last uses $s_i = r_i(\vec{v})$ at the solved curve, so the matrix being inverted is the instrument sensitivity jacobian $\frac{\partial r_i}{\partial v_j}$ derived below. The inverse is of that whole matrix rather than of each entry, so $\left(\frac{\partial s_i}{\partial v_j} \right)^{-1}$ should be read as the (j, i) entry of the inverted jacobian. Note this assumes the calibration is square and fits exactly. For the regularised solver of the constrained optimization section, where nodes outnumber instruments, $\frac{\partial v_j}{\partial s_i}$ comes instead from the response of the KKT system to a quote bump, derived at the end of that section.

The use of the chain rule separates the instrument sensitivities (to dfs) from the pv sensitivity (to dfs) and allows us to calculate them separately. The sensitivities to dfs $\left(\frac{\partial \Pi}{\partial v_j} \right)$ are discussed later.

With electronic trading, splitting the risk calculation into two parts is useful since the instrument sensitivities can be pre-calculated and stored, and then the pv sensitivities can be calculated on the fly for any portfolio.

1.1.2 Forward Risk

The DV01 above expresses risk against tradeable instruments s_i . In practice we also want risk buck-eted against the curve's own forward rates — it doesn't depend on a choice of hedging instruments, so it survives changes to the calibration set.

$$R_i^{\text{fwd}} = \frac{1}{10,000} \frac{\partial \Pi}{\partial f_i}$$

(scaling since forwards quoted in natural units)

Again use the chain rule, splitting into the reflows $\frac{\partial \Pi}{\partial v_j}$ derived below and the curve's own sensitivity to its forwards:

$$\begin{aligned} R_i^{\text{fwd}} &= \frac{1}{10,000} \frac{\partial \Pi}{\partial f_i} \\ &= \frac{1}{10,000} \frac{\partial \Pi}{\partial v_j} \frac{\partial v_j}{\partial f_i} \end{aligned}$$

With continuously compounded forward & log-linear interpolation where forwards f_i over interval (T_i, T_{i+1}) are constant, we have discount factor as:

$$v_j = \exp(-f_k \tau_{jk})$$

(summation inside exponent due to repeated index in k)

Differentiating with respect to a single forward f_i , the dummy k is summed against $\frac{\partial f_k}{\partial f_i} = \delta_{ki}$, which collapses the sum to its $k = i$ term:

$$\begin{aligned} \frac{\partial v_j}{\partial f_i} &= -\tau_{jk} \frac{\partial f_k}{\partial f_i} v_j \\ &= -\tau_{jk} \delta_{ki} v_j \\ &= -\tau_{ji} v_j \end{aligned}$$

Note τ_{jk} depends only on dates, not on curve levels, so it passes through the derivative untouched.

And so we have:

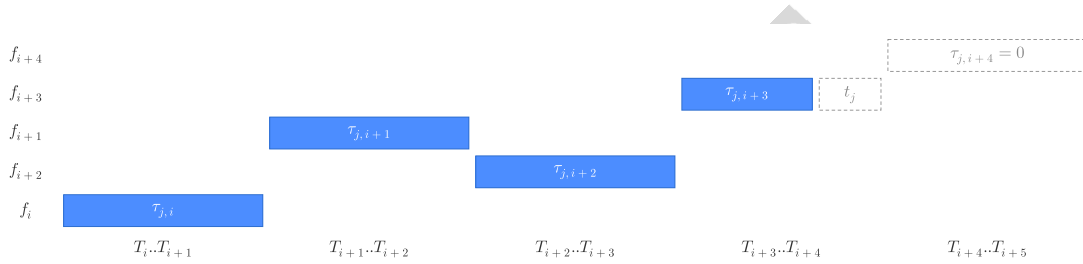
$$\begin{aligned} R_i^{\text{fwd}} &= \frac{1}{10,000} \frac{\partial \Pi}{\partial v_j} \frac{\partial v_j}{\partial f_i} \\ &= -\frac{1}{10,000} \frac{\partial \Pi}{\partial v_j} \tau_{ji} v_j \end{aligned}$$

τ_{ji} is the year fraction of the overlap of $(0, t_j)$ with the pillar interval, and j indexes cashflow dates:

$$\tau_{ji} = \frac{1}{365.25} \begin{cases} T_{i+1} - T_i & T_{i+1} \leq t_j \\ t_j - T_i & T_i < t_j < T_{i+1} \\ 0 & t_j \leq T_i \end{cases}$$

so a cashflow accrues a full interval for every pillar it has passed, part of the one it lands in, and none of those beyond it.

Here j runs over cashflow dates, so $\frac{\partial \Pi}{\partial v_j} = c_j$ and no interpolation onto nodes is needed. Equivalently one may take j over nodes, using the interpolated reflows of the next section, in which case τ_{ji} is always a full interval or zero. The two agree because log-linear interpolation is exactly piecewise-constant forwards, so the partial τ_{ji} and the interpolation weight w_2 are the same year-fraction split.



Each bar is a forward held constant across its own interval, and its width is the τ_{ji} that forward contributes to t_j : full for the intervals already passed, cut short by t_j in the one it lands in, and absent beyond. Each bar's vertical position is schematic — the forward curve rises and falls, and the diagram has no axis to show that beyond the row order. Every bar therefore contributes one factor to the discount,

$$v_j = \prod_i e^{-f_i \tau_{ji}}$$

which is exact, not an approximation — it is what piecewise-constant forwards mean.

The portfolio value is additive over any partition of its trades, $\frac{\partial \Pi}{\partial v_j} = \sum_b \frac{\partial \Pi^{(b)}}{\partial v_j}$, so the same contraction gives per-book or per-deal forward risk whose rows sum back to the portfolio total.

1.1.3 Representation Flows $\frac{\partial \Pi}{\partial v_j}$

The sensitivity $\frac{\partial \Pi}{\partial v_j}$ shared by both chains above, also known as reflows (Representation Flows), is similar to cashflows.

We can express the portfolio reflows in terms of instrument reflows:

$$\frac{\partial \Pi}{\partial v_j} = w_i \frac{\partial \pi_i}{\partial v_j}$$

and for an individual instrument reflow:

$$\frac{\partial \pi_i}{\partial v_j} = \frac{\partial(c_{ij}v_{ij})}{\partial v_j}$$

for simplification, let's drop the instrument index i and just look at a single instrument. Then we can express the instrument reflow as:

$$\begin{aligned}\frac{\partial \pi_i}{\partial v_j} &= \frac{\partial(c_{ij}v_{ij})}{\partial v_j} \\ \frac{\partial \pi}{\partial v_j} &= \frac{\partial(c_k v_k)}{\partial v_j} \\ &= c_k \delta_{kj} \\ &= c_j\end{aligned}$$

where δ_{kj} is the Kronecker delta, which is 1 if $k = j$ and 0 otherwise, collapsing the sum over cashflow dates to its $k = j$ term.

Consider case where cashflow falls on node date. Note here we are looking at a single component without summing out:

$$\begin{aligned}\frac{\partial \pi}{\partial v_i} &= \frac{\partial(c_i v_i)}{\partial v_i} \\ &= c_i\end{aligned}$$

Consider case where cashflow does not fall on node date:

We can see $v(t) = v(t; v_j, v_{j+1})$, interpolated between the two nodes bracketing cashflow date t , so $T_j < t < T_{j+1}$ where T_j are the node dates.

$$\begin{aligned}\frac{\partial \pi}{\partial v_j} &= \frac{\partial \pi}{\partial v(t)} \frac{\partial v(t)}{\partial v_j} \\ \frac{\partial \pi}{\partial v_{j+1}} &= \frac{\partial \pi}{\partial v(t)} \frac{\partial v(t)}{\partial v_{j+1}}\end{aligned}$$

Log linear interpolation Using the log-linear interpolation scheme:

$$\begin{aligned}\ln v(t) &= w_1 \ln v_j + w_2 \ln v_{j+1} \\ w_1 &= \frac{T_{j+1} - t}{T_{j+1} - T_j}, \quad w_2 = \frac{t - T_j}{T_{j+1} - T_j}\end{aligned}$$

Since the weights w_1, w_2 depend only on dates and not curve levels, differentiating gives:

$$\frac{\partial v(t)}{\partial v_j} = w_1 \frac{v(t)}{v_j}, \quad \frac{\partial v(t)}{\partial v_{j+1}} = w_2 \frac{v(t)}{v_{j+1}}$$

so by the chain rule, the cashflow's reflow splits across its two bracketing nodes:

$$\begin{aligned}\frac{\partial \pi}{\partial v_j} &= \frac{\partial \pi}{\partial v(t)} \frac{\partial v(t)}{\partial v_j} = c \frac{\partial v(t)}{\partial v_j} \\ \frac{\partial \pi}{\partial v_{j+1}} &= \frac{\partial \pi}{\partial v(t)} \frac{\partial v(t)}{\partial v_{j+1}} = c \frac{\partial v(t)}{\partial v_{j+1}}\end{aligned}$$

1.1.4 Auto differentiation

The above breaks down in some cases (e.g. if $c_i = c_i(v_i)$).

1.1.5 Instrument Sensitivities $\frac{\partial r_i}{\partial v_j}$

The above is a jacobian relating market instrument sensitivities to nodes. Various ways to calculate from bumping or automatic differentiation.

1.1.6 Curve solver

Require a discount factor state that returns the discount factor at time t_i , that is $v(t_i)$. Typically, we have various instruments with corresponding market valuations s_i . We create a model valuation function $r_i(v)$ that fits the market valuation points and interpolates in between (using say log linear or plain linear).

With an objective function $g(\vec{v}) = (r_i - s_i)W_{ij}(r_j - s_j)$ where each s_i are the fixed market valuations and the r_i are dependent on the underlying curve $v(t_i)$.

In general we start with change between iterations:

$$\Delta v_i = v'_i - v_i$$

Gradient descent We can iterate to find the optimal curve $v(t_i)$ that minimizes the objective function $g(v_i)$ using:

Derivation

To ensure “descent” we need $g(\vec{v}') < g(\vec{v})$.

Estimate 1st order Taylor expansion of g around v_k :

$$g(v') \approx g(v) + \frac{\partial g}{\partial v_i} \Delta v_i < g(v)$$

and thus we require:

$$\frac{\partial g}{\partial v_i} \Delta v_i < 0$$

To ensure this, we can set Δv_i to be a negative multiple of the gradient:

$$\Delta v_i = -\alpha \frac{\partial g}{\partial v_i}$$

where α is a step size. This gives the update rule:

$$v'_i = v_i - \alpha \frac{\partial g}{\partial v_i}$$

where α is a step size.

Set $x_i = r_i - s_i$ as residual. We want to determine how r_i changes along negative gradient $-\frac{\partial g}{\partial v_j}$. Set directional derivative as:

$$y_i = \frac{\partial r_i}{\partial v_j} \frac{\partial g}{\partial v_j}$$

We can approximate change in x_i along the negative gradient as:

$$x'_i = r'_i - s_i \approx x_i - \alpha y_i$$

where α is a step size.

Substituting back in g to give as function of α :

$$g(\alpha) = (x_i - \alpha y_i) W_{ij} (x_j - \alpha y_j)$$

Minimizing $g(\alpha)$ with respect to α gives the optimal step size along the negative gradient direction. Taking derivative and setting to zero:

Derivation

$$\begin{aligned} g(\alpha) &= (x_i - \alpha y_i) W_{ij} (x_j - \alpha y_j) \\ &= x_i W_{ij} x_j - \alpha y_i W_{ij} x_j - \alpha x_i W_{ij} y_j + \alpha^2 y_i W_{ij} y_j \\ &= x_i W_{ij} x_j - 2\alpha y_i W_{ij} x_j + \alpha^2 y_i W_{ij} y_j \quad (\text{since } W_{ij} \text{ is symmetric}) \\ \frac{\partial g}{\partial \alpha} &= -2y_i W_{ij} x_j + 2\alpha y_i W_{ij} y_j = 0 \end{aligned}$$

$$\alpha = \frac{x_i W_{ij} y_j}{y_i W_{ij} y_j}$$

Gauss-Newton Look at expansion of r_i around v_k :

$$\begin{aligned} r_i(v') &= r_i(v + \Delta v) \\ &\approx r_i(v) + \frac{\partial r_i}{\partial v_j} \Delta v_j \end{aligned}$$

Substituting into the objective function $g(v')$ gives:

$$\begin{aligned} g(v') &= (r_i(v') - s_i)W_{ij}(r_j(v') - s_j) \\ &\approx (r_i(v) + \frac{\partial r_i}{\partial v_m} \Delta v_m - s_i)W_{ij}(r_j(v) + \frac{\partial r_j}{\partial v_l} \Delta v_l - s_j) \\ &= (x_i + \frac{\partial r_i}{\partial v_m} \Delta v_m)W_{ij}(x_j + \frac{\partial r_j}{\partial v_l} \Delta v_l) \\ &= x_i W_{ij} x_j + 2x_i W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l + \Delta v_m \frac{\partial r_i}{\partial v_m} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l \end{aligned}$$

where $x_i = r_i(v_k) - s_i$ is the residual at the current iterate.

Note that:

$$\begin{aligned} \frac{\partial g}{\partial v_k} &= \frac{\partial(r_i - s_i)}{\partial v_k} W_{ij}(r_j - s_j) + (r_i - s_i)W_{ij} \frac{\partial(r_j - s_j)}{\partial v_k} \\ &= \frac{\partial r_i}{\partial v_k} W_{ij}(r_j - s_j) + (r_i - s_i)W_{ij} \frac{\partial r_j}{\partial v_k} \\ &= 2 \frac{\partial r_i}{\partial v_k} W_{ij}(r_j - s_j) \\ &= 2 \frac{\partial r_i}{\partial v_k} W_{ij} x_j \end{aligned}$$

So:

$$g(v') \approx x_i W_{ij} x_j + \frac{\partial g}{\partial v_l} \Delta v_l + \Delta v_m \frac{\partial r_i}{\partial v_m} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l \quad (\text{GN.a})$$

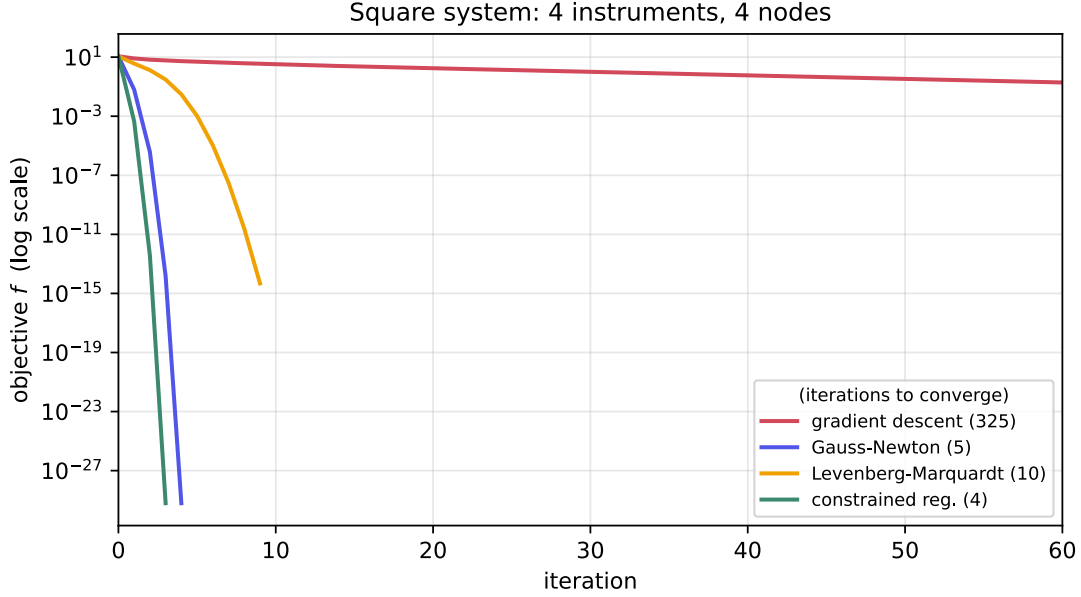
and

$$\begin{aligned} \frac{\partial g(v')}{\partial v'_k} &= \frac{\partial g(v')}{\partial \Delta v_k} \\ &= 0 + \frac{\partial g}{\partial v_k} + 2 \frac{\partial r_i}{\partial v_k} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l \end{aligned} \quad (\text{GN.b})$$

Set the derivative to zero to find the optimal Δv :

$$\frac{\partial r_i}{\partial v_k} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l = -\frac{1}{2} \frac{\partial g}{\partial v_k}$$

where the lhs applies a matrix to Δv_l , so both sides are vectors. This is a linear system of equations that can be solved for Δv_l .



A square system — 4 swaps against 4 free nodes — so every method reaches the same curve and only the path differs. Gradient descent needs 325 iterations to drive g to 10^{-9} : each step is only the locally steepest direction, and the optimal α above rescales that direction without redirecting it. Gauss-Newton uses the curvature carried by $\frac{\partial r_i}{\partial v_j}$ and converges in 5, at machine precision. Levenberg-Marquardt and constrained optimization are plotted for reference and are covered below.

Levenberg-Marquardt Combines the above 2 methods by adding a damping term $\lambda \Delta v_m \delta_{ml} \Delta v_l$ to the objective function (a quadratic penalty term).

Set objective function as :

$$g_{LM}(\Delta v) = g(v') + \lambda \Delta v_m \delta_{ml} \Delta v_l$$

Then from (GN.a),

$$g(v') \approx x_i W_{ij} x_j + \frac{\partial g}{\partial v_l} \Delta v_l + \Delta v_m \frac{\partial r_i}{\partial v_m} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l$$

and (GN.b),

$$\frac{\partial g(v')}{\partial \Delta v_k} = \frac{\partial g}{\partial v_k} + 2 \frac{\partial r_i}{\partial v_k} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l$$

For additional term,

$$\begin{aligned} \frac{\partial}{\partial \Delta v_k} &= \lambda \left(\frac{\partial \Delta v_m}{\partial \Delta v_k} \delta_{ml} \Delta v_l + \Delta v_m \delta_{ml} \frac{\partial \Delta v_l}{\partial \Delta v_k} \right) \\ &= \lambda (\delta_{mk} \delta_{ml} \Delta v_l + \Delta v_m \delta_{ml} \delta_{lk}) \\ &= \lambda (\delta_{kl} \Delta v_l + \Delta v_m \delta_{mk}) \\ &= 2\lambda \delta_{kl} \Delta v_l \end{aligned}$$

Set above to zero and solve for the step:

$$\frac{\partial g_{LM}}{\partial \Delta v_k} = \frac{\partial g}{\partial v_k} + 2 \frac{\partial r_i}{\partial v_k} W_{ij} \frac{\partial r_j}{\partial v_l} \Delta v_l + 2\lambda \delta_{kl} \Delta v_l = 0$$

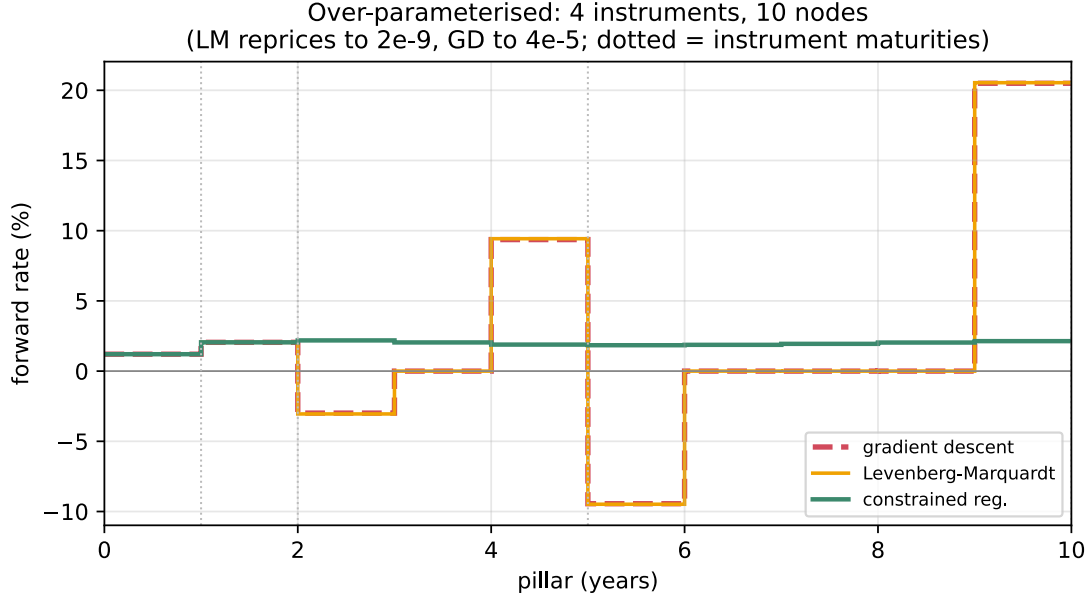
Factor out step:

$$\left(\frac{\partial r_i}{\partial v_k} W_{ij} \frac{\partial r_j}{\partial v_l} + \lambda \delta_{kl} \right) \Delta v_l = -\frac{1}{2} \frac{\partial g}{\partial v_k}$$

Note that λ dynamically shifts the solver's behavior during iterations:

- As $\lambda \rightarrow 0$, the damping term vanishes, and the system exactly recovers the Gauss-Newton update.
- As $\lambda \rightarrow \infty$, the diagonal penalty dominates the left-hand side, reducing the system to $\lambda \delta_{kl} \Delta v_l \approx -\frac{1}{2} \frac{\partial g}{\partial v_k}$. This simplifies to $\Delta v_k \approx -\frac{1}{2\lambda} \frac{\partial g}{\partial v_k}$, which matches update rule for gradient descent with step size of $\alpha = \frac{1}{2\lambda}$.

Constrained Optimization In earlier unconstrained optimization, the objective function weights each instrument's residual squares so you might weight liquid swaps say greater. In general you will have more nodes than instruments so weighted least squares might be under-determined.



The same 4 swaps against 10 annual nodes. The Jacobian $\frac{\partial r_i}{\partial v_j}$ is 10×4 with rank 4, so 6 node directions are unconstrained by the data. Gauss-Newton fails outright, its normal matrix being singular. Gradient descent and Levenberg-Marquardt do return a curve, and it reprices every swap (Levenberg-Marquardt to 2×10^{-9} , gradient descent to 4×10^{-5}), but it is arbitrary: forwards swing between -9.5% and $+20.5\%$ along the unconstrained directions. Nothing in the unconstrained objective prefers one point of that null space over another, and supplying that preference is exactly what the penalty matrix below does.

In constrained optimization, the objective function takes a different form where penalty matrix P_{ij} is applied to each iterative state delta. The objective function becomes a regularization prior on the nodes, minimizing curve “roughness” or deviation from previous state.

That is, we minimize the following objective function:

$$g(\Delta \ln v) = \frac{1}{2} P_{ij} \Delta \ln v_i \Delta \ln v_j$$

Also the change in iterations is now in forward (log) space, so the increment is a log increment $\Delta \ln v_i$ rather than the level increment $\Delta v_i = v'_i - v_i$ used in the unconstrained methods above:

$$v'_i = v_i e^{\Delta \ln v_i}$$

From the forward risk section, $\ln v_j = -f_k \tau_{jk}$, so forwards are linear in log discount factors:

$$\Delta \ln v_j = -\tau_{jk} \Delta f_k$$

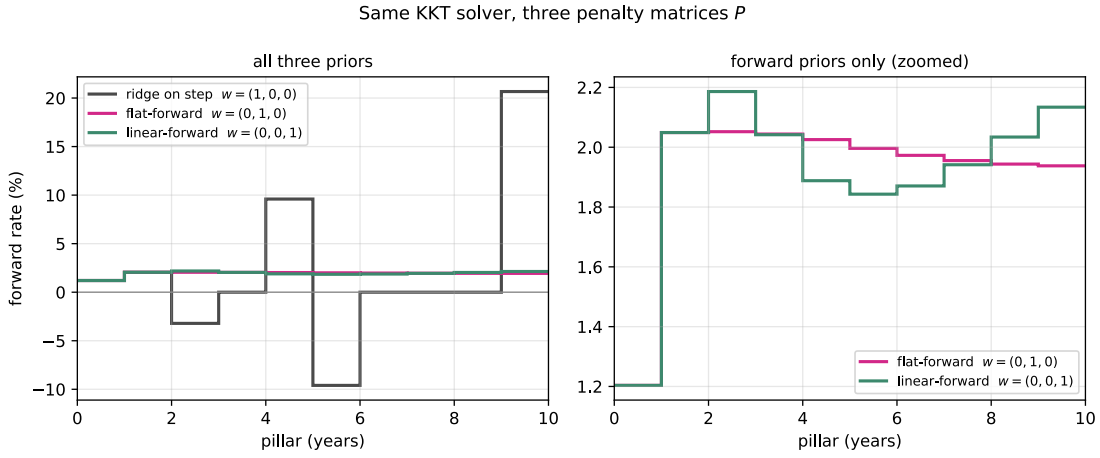
P_{ij} is normally built as a prior on forward curve levels.

The map is exact rather than a linearization, the prior carries no linearization error, and since it acts on forward levels it does not scale with interval length or discount factor level, neither of which would hold for a penalty applied to Δv_i directly. The exponential update also keeps every discount factor strictly positive whatever step is taken.

Writing the constant map as $\Delta f = F \Delta \ln v$ and the first and second difference operators on adjacent intervals as D_1 and D_2 , the penalty used below is a weighted blend

$$P = w_0 I + w_1 (D_1 F)^\top (D_1 F) + w_2 (D_2 F)^\top (D_2 F)$$

with weights $w = (w_0, w_1, w_2)$: a ridge on the step itself, a flat-forward prior and a linear-forward prior respectively.



Same 4 swaps, same 10 nodes, same KKT / CO solver (derived below) with only P differing. All three penalties reprice to 10^{-14} . The penalty names come from their respective matrix null spaces (hence the shape the curve relaxes to when data says nothing). A ridge on the step, $w = (1, 0, 0)$, has trivial null space: it penalises the size of $\Delta \ln v$ and nothing about its shape, so it stays as ragged as Levenberg-Marquardt, with roughness $\sum_i (\Delta^2 f)_i^2 = 2327$. Pulling the penalty back through F is what smooths: first differences leave constant forwards free, giving 0.71, and second differences leave affine forwards free, giving 0.60. The KKT system on its own buys none of this.

Require the updated state v'_i to price instruments exactly. Since r_i is nonlinear this can only hold at convergence; each step enforces the linearised version, as in sequential quadratic programming. So we have constraints:

$$\begin{aligned}
r_i(v') - s_i &= r_i(v e^{\Delta \ln v}) - s_i \\
&\approx r_i(v) + \sum_j v_j \frac{\partial r_i}{\partial v_j} \Delta \ln v_j - s_i \\
&= r_i(v) - s_i + \sum_j v_j \frac{\partial r_i}{\partial v_j} \Delta \ln v_j \\
&= x_i + \sum_j v_j \frac{\partial r_i}{\partial v_j} \Delta \ln v_j \\
&= 0
\end{aligned}$$

where the approximation keeps only first order in $\Delta \ln v$, since $v_j e^{\Delta \ln v_j} - v_j \approx v_j \Delta \ln v_j$. Note the sum over j is written explicitly here: j appears three times in $v_j \frac{\partial r_i}{\partial v_j} \Delta \ln v_j$, so the summation convention used elsewhere in this document cannot apply. Below we absorb the v_j into a log derivative so the convention can be restored.

And so

$$\sum_j v_j \frac{\partial r_i}{\partial v_j} \Delta \ln v_j = -x_i$$

Since $v_j \frac{\partial r_k}{\partial v_j} = \frac{\partial r_k}{\partial \ln v_j}$ for each j , the constraint is

$$\frac{\partial r_k}{\partial \ln v_j} \Delta \ln v_j = -x_k$$

$\frac{\partial r_k}{\partial \ln v_j}$ is the sensitivity of instrument k to log discount factor j .

This is used as an equality constraint in the optimization problem. Expressed as a Lagrangian, we have:

$$\mathcal{L}(\Delta \ln v, \lambda) = \frac{1}{2} P_{ij} \Delta \ln v_i \Delta \ln v_j + \lambda_k \left(\frac{\partial r_k}{\partial \ln v_j} \Delta \ln v_j + x_k \right)$$

To minimize the Lagrangian, we take derivatives with respect to $\Delta \ln v_i$ and λ_k and set them to zero:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial \Delta \ln v_i} &= P_{ij} \Delta \ln v_j + \frac{\partial r_k}{\partial \ln v_i} \lambda_k = 0 & (P \text{ symmetric}) \\
\frac{\partial \mathcal{L}}{\partial \lambda_k} &= \frac{\partial r_k}{\partial \ln v_j} \Delta \ln v_j + x_k = 0
\end{aligned}$$

In matrix form, we can express this as a block matrix system:

$$\begin{bmatrix} P_{ij} & \frac{\partial r_k}{\partial \ln v_i} \\ \frac{\partial r_k}{\partial \ln v_j} & 0 \end{bmatrix} \begin{bmatrix} \Delta \ln v_j \\ \lambda_k \end{bmatrix} = \begin{bmatrix} 0 \\ -x_k \end{bmatrix}$$

Reading the two block rows back recovers the stationarity conditions above, with free indices i and k respectively. The upper-right block is the same object as the lower-left with its indices swapped, which is all a transpose means in index notation.

In practice the solved step is damped: the update is applied as $v'_i = v_i e^{\alpha \Delta \ln v_i}$ with α backtracked from 1 until the repricing merit actually improves. The α sits only on the update, not in the system above, which is solved for the undamped $\Delta \ln v_i$.

This is a special case of KKT conditions [2] for equality constraints. The solution to this system gives the optimal $\Delta \ln v_j$ that minimizes the objective function while satisfying the constraints. The general case can also include inequality constraints, which can be handled using methods like interior point or active set methods.

The penalty matrix P is not necessarily positive definite; with $w_0 = 0$, it is singular. The operator $D_1 F$ maps parallel shifts to zero, and $D_2 F$ maps affine forwards to zero, meaning these unpenalized shapes span the null space $\ker P$.

The block system is nonsingular if the instrument sensitivities $\frac{\partial r_k}{\partial \ln v_j}$ are linearly independent and no non-zero element of $\ker P$ leaves all instruments unchanged. Every shape that receives zero penalty from the prior must be constrained by at least one instrument. A flat-forward prior requires at least one instrument sensitive to a parallel shift, and a linear-forward prior requires two instruments of different maturities.

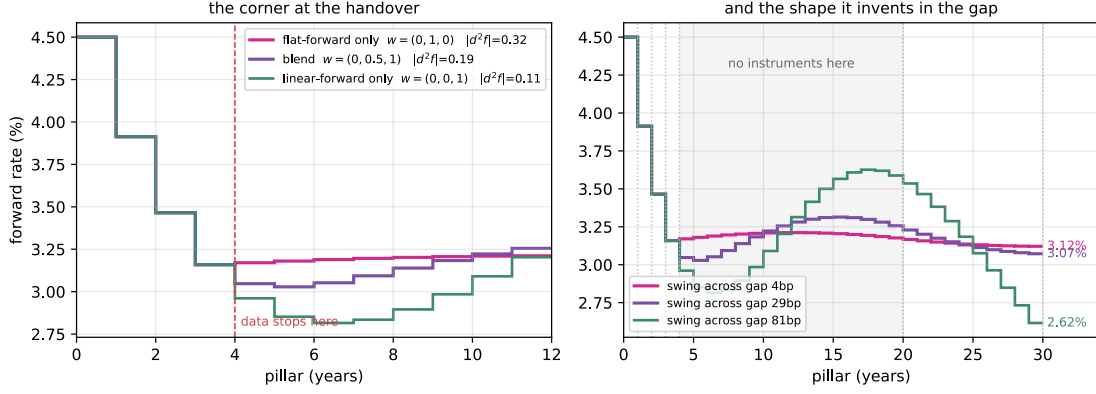
If this condition fails—such as a single swap calibrated with a pure D_2 prior—the system is singular, which is resolved by adding a small ridge penalty $w_0 > 0$. The curve takes shapes from $\ker P$ only in the directions that the instruments leave unconstrained.

The same system supplies the $\frac{\partial v_j}{\partial s_i}$ that the DV01 section needed for a non-square calibration. At the solved curve $x_k = 0$, so a bump δs_k to the quotes enters as $x_k \rightarrow -\delta s_k$, and the step it produces is the curve's response:

$$\begin{bmatrix} P_{ij} & \frac{\partial r_k}{\partial \ln v_i} \\ \frac{\partial r_k}{\partial \ln v_j} & 0 \end{bmatrix} \begin{bmatrix} \frac{\partial \ln v_j}{\partial s_l} \\ \frac{\partial \lambda_k}{\partial s_l} \end{bmatrix} = \begin{bmatrix} 0 \\ \delta_{kl} \end{bmatrix}$$

with $\frac{\partial v_j}{\partial s_l} = v_j \frac{\partial \ln v_j}{\partial s_l}$ (no sum over j). This is the implicit function theorem applied to the KKT conditions. Note that with more nodes than instruments the solved curve is not a function of the quotes alone, since it depends on where the iteration started, so this is the minimum- P -norm response to a bump rather than the derivative of a uniquely defined map. In the square case with P dropped it reduces to the inverse jacobian used in the DV01 section.

Blending the priors: 6 instruments, 30 nodes, 4Y-20Y gap
(all three reprice every instrument exactly)



Neither forward prior dominates. Six swaps out to 30Y with a 4Y-20Y gap and 30 annual nodes; all three runs reprice every instrument to 10^{-14} , so none of the difference below is misfit. Left, flat-forward alone has to turn a corner where the data stops, since constant forwards is the only shape free to it — $\max |\Delta^2 f| = 0.32$ against 0.11 for linear-forward. Right, linear-forward fails the other way: affine forwards being free, it runs the front end's downward slope straight through the empty gap, swinging 81bp where flat-forward swings 4bp and finishing 50bp lower at 30Y. The blend $w = (0, 0.5, 1)$ softens the corner to 0.19 while holding the swing to 29bp.

Note that every curve plotted in these figures is an instantaneous continuously compounded forward, $f_i = (\ln v_i - \ln v_{i+1})/\Delta t_i$, constant across each interval by the log linear construction above — the steps are the actual shape, not a plotting artefact. The quoted swap rates are par rates and appear only as the dotted maturity lines. Since a par rate is an annuity weighted average over the whole forward path, on a downward sloping curve the far forward sits below the par rate of the same maturity even under an exact fit.

The four solver figures are produced by `make_figures.py` in this directory, which solves each case with `flib.rates.curve.SolvedCurve` and prints the underlying tables.

Par Rate Jacobian with Residual Attributions Calibration equation for curve depends on both curve parameters P' and current curve parameters P and ensures that the model exactly matches fixed market quotes s_a for that layer:

$$s_a = r_a(P', P)$$

Apply multivariate chain rule:

$$\begin{aligned} \frac{\partial r_a}{\partial P'_b} \frac{\partial P'_b}{\partial s_m} + \frac{\partial r_a}{\partial P_j} \frac{\partial P_j}{\partial s_m} &= \frac{\partial s_a}{\partial s_m} \\ &= \delta_{am} \\ &= 0 \end{aligned}$$

Apply inverse function theorem on Jacobian. Product of inverse Jacobian with itself contracts to the Kronecker delta δ_{ij}

$$\begin{aligned}\left(\frac{\partial r_i}{\partial P_a}\right)^{-1} \frac{\partial r_a}{\partial P'_b} \frac{\partial P'_b}{\partial s_m} + \left(\frac{\partial r_i}{\partial P_a}\right)^{-1} \frac{\partial r_a}{\partial P_j} \frac{\partial P_j}{\partial s_m} &= 0 \\ \left(\frac{\partial r_i}{\partial P_a}\right)^{-1} \frac{\partial r_a}{\partial P'_b} \frac{\partial P'_b}{\partial s_m} + \delta_{ij} \frac{\partial P_j}{\partial s_m} &= 0 \\ \left(\frac{\partial r_i}{\partial P_a}\right)^{-1} \frac{\partial r_a}{\partial P'_b} \frac{\partial P'_b}{\partial s_m} + \frac{\partial P_i}{\partial s_m} &= 0\end{aligned}$$

Rearrange terms isolates current layer's parameter sensitivity, establishes exact recursive step to propagate risk from underlying curve P' to current curve P :

$$\frac{\partial P_i}{\partial s_m} = - \left(\frac{\partial r_i}{\partial P_a}\right)^{-1} \frac{\partial r_a}{\partial P'_b} \frac{\partial P'_b}{\partial s_m}$$

To unroll the recursion, let's explicitly add layer superscripts. Let N be the current dependent layer, $N-1$ be the underlying layer, and 1 be the base layer. The base layer market quote is $s_m^{(1)}$.

The recursive step for layer N is:

$$\frac{\partial P_i^{(N)}}{\partial s_m^{(1)}} = - \left(\frac{\partial r_i^{(N)}}{\partial P_a^{(N)}}\right)^{-1} \frac{\partial r_a^{(N)}}{\partial P_b^{(N-1)}} \frac{\partial P_b^{(N-1)}}{\partial s_m^{(1)}}$$

Step 1: Unroll the sensitivity for layer $N-1$

We apply the exact same recursive logic to the underlying layer $N-1$, which depends on layer $N-2$:

$$\frac{\partial P_b^{(N-1)}}{\partial s_m^{(1)}} = - \left(\frac{\partial r_b^{(N-1)}}{\partial P_c^{(N-1)}}\right)^{-1} \frac{\partial r_c^{(N-1)}}{\partial P_d^{(N-2)}} \frac{\partial P_d^{(N-2)}}{\partial s_m^{(1)}}$$

Substitute this back into the layer N equation. We group the terms in brackets to highlight the local transition matrix for each layer:

$$\frac{\partial P_i^{(N)}}{\partial s_m^{(1)}} = \left[- \left(\frac{\partial r_i^{(N)}}{\partial P_a^{(N)}}\right)^{-1} \frac{\partial r_a^{(N)}}{\partial P_b^{(N-1)}} \right] \left[- \left(\frac{\partial r_b^{(N-1)}}{\partial P_c^{(N-1)}}\right)^{-1} \frac{\partial r_c^{(N-1)}}{\partial P_d^{(N-2)}} \right] \frac{\partial P_d^{(N-2)}}{\partial s_m^{(1)}}$$

Step 2: Unroll down to the base layer ($k=1$)

If we assume a three-layer hierarchy ($N=3$), the next substitution hits the base layer. The sensitivity of the base layer parameters $P^{(1)}$ to their own market quotes $s^{(1)}$ is simply the inverse of the base layer Jacobian:

$$\frac{\partial P_y^{(1)}}{\partial s_m^{(1)}} = \left(\frac{\partial r_y^{(1)}}{\partial P_m^{(1)}} \right)^{-1}$$

Substituting this terminal case into the chain gives the fully unrolled three-layer cascade:

$$\frac{\partial P_i^{(3)}}{\partial s_m^{(1)}} = \left[- \left(\frac{\partial r_i^{(3)}}{\partial P_a^{(3)}} \right)^{-1} \frac{\partial r_a^{(3)}}{\partial P_b^{(2)}} \right] \left[- \left(\frac{\partial r_b^{(2)}}{\partial P_c^{(2)}} \right)^{-1} \frac{\partial r_c^{(2)}}{\partial P_y^{(1)}} \right] \left(\frac{\partial r_y^{(1)}}{\partial P_m^{(1)}} \right)^{-1}$$

TBD - how to use par rate jacobians to calculate attribution of market quote sensitivities through the layered curve structure.

2 References

- [1] Darbyshire, J. H. M. (2017). Pricing and Trading Interest Rate Derivatives: A Practical Guide to Swaps. Aitch & Dee Limited. <https://books.google.co.uk/books?id=gOx3tAEACAAJ>
- [2] Kuhn, H. W., & Tucker, A. W. (1951). NONLINEAR PROGRAMMING.